Program 3: Persistent Storage

Aaron Quashnock

Note: The game I am considering this storage solution for is different from the game shown in the demo, as I have switched the idea for my game. The game I am considering a storage solution for in this write up is the roguelite carnival shooter that I wrote my proposal about. I decided to prototype and test the storage solution with my old game for this program, as it is what I currently have the most progress on, but I do plan to implement this storage solution in my full game as well. I have had this approach approved by the professor.

Relational Database

I have some prior experience with relational databases and know that they can hold a large amount of data and be parsed quickly and efficiently with SQL or another query language. I considered this solution as my game would keep track of various stats of the player such as their highest score and used bullets as they play over a long period of time. However, I decided against this approach because the database server would likely need to be run as a separate instance while playing or need to run remotely. The game is single player and so requiring an internet connection to play should be avoided. Additionally, while the ability to store growing data over time is helpful, the data that is stored by my game can be made finite through other methods.

Custom File Format

Since my game has a lot of specific concerns with how to save and retrieve data, I also considered a custom file format for storing data. Since certain data, such as the current upgrades purchased and money, would need to be quickly saved and retrieved, I could set up the file in such a way that retrieval of that data could be prioritized. However, I also decided against this approach because my game would only contain one scene, and so I would not need to structure a file such that it could be duplicated for all scenes. Additionally, creating a structure for reading and writing data takes time when there are already common formats for reading and writing to a file with supported methods and libraries already written.

Serialized and Encrypted JSON

Finally, I considered JSON for storage as I had also worked with it previously in web development and was already familiar with its structure. The key-value format fits many of the applications of storage I had in mind for the game, such as keeping track of the player's upgrades and money while having existing support in many Unity libraries and packages.

However, some of the data that I had in mind like running data would best be represented internally using custom objects with specific fields for the data, such as the final score and the bullets used throughout the run. This complex data can be stored using object serialization to represent objects within the JSON format. I experimented with this in the demo for Program 3 using serialization to express a dictionary object for storing which enemies had been defeated. Serialized run data can be formatted as objects, stored and then retrieved to show players a log of their previous runs.

Finally, I was concerned with data manipulation by the user, as the JSON file would be stored locally on their computer. While the game is not multiplayer and manipulating data in this way would not directly harm the experience of other players, the game's core loop surrounds getting the highest score in a run you can. The ability to manipulate values internally could be used to invalidate the core gameplay loop. To fix this issue, I decided to use encryption so that the player could not easily manipulate the data while allowing it to still be written and read by the game. Since the encryption algorithm would be stored locally along with the game, the user could still discover the encryption algorithm and use it to manipulate their data. However, the effort and technical skill required to discover the encryption algorithm this way should still deter players from manipulating the data and invalidating their runs. I have also implemented a basic encryption algorithm in the Program 3 demo.

For these reasons, I have decided to choose a serialized and encrypted JSON storage solution for my game.